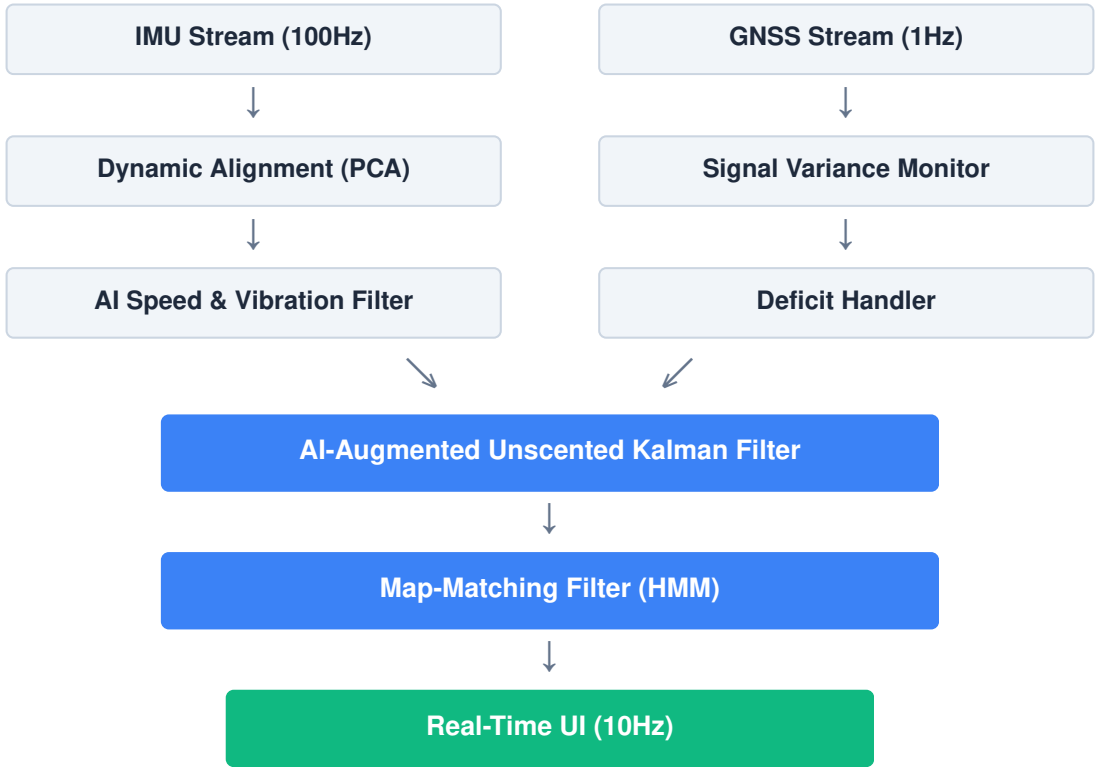
Intelligent Dead Reckoning (IDR)

Execution Plan, Task Prompts & API Contracts

> **Target Performance Benchmark:** Drift must be strictly contained to <10% of total distance traveled in GNSS-denied environments. High-frequency (10Hz+) updates are required on edge hardware.

1. System Architecture Pipeline



2. Hackathon Task Distribution (Prompt-Ready)

Task 1: AI & Kinematic Modeling (Deep Learning)

- **Objective:** Build a PyTorch pipeline taking a `(Batch, 6, 20)` tensor of accelerometer and gyroscope data and outputting a continuous `[velocity, yaw_rate]` prediction.
- **Architecture:** Implement a 1D-CNN for initial noise filtration to drop sudden pothole impulses, feeding into a Temporal Convolutional Network (TCN) to map historical acceleration to velocity in O(1) parallel time.
- **Loss Function:** Code a multi-objective loss combining standard MSE against the ground-truth speed and a physics-informed penalty enforcing the kinematic constraint (v_t = v_{t-1} + a_t * dt).

Task 2: State Estimation & Map Matching (Algorithms)

- **Objective:** Write the Unscented Kalman Filter (UKF) and Hidden Markov Model (HMM) to fuse the high-frequency AI predictions with low-frequency 1Hz GNSS data.
- **Sensor Fusion:** Implement the Unscented Transform, explicitly calculating sigma points to handle the non-linear kinematics of the vehicle without linearizing via Jacobians.
- **Dynamic Programming:** Code the Viterbi algorithm for the HMM to snap the UKF coordinates to the offline OSM road grid, strictly optimizing time complexity to prevent blocking the main inference loop.

Task 3: Edge Execution & Architecture (Systems)

- **Objective:** Construct a low-latency C++ ingestion engine that orchestrates the asynchronous data streams, structuring the pipeline like a quantitative execution engine.
- **Model Deployment:** Quantize the trained PyTorch TCN to INT8, export it to ONNX, and write the C++ inference wrapper to execute the graph directly on edge hardware.
- **Concurrency:** Build memory-safe, lock-free circular buffers to ingest the 100Hz IMU and 1Hz GNSS independently, routing the C++ structs to the UKF without thread contention.

Task 4: Mobile Interface & UI (Frontend)

- **Objective:** Develop the native Android or iOS application that acts as the hardware polling layer and final visualization interface.
- **Hardware Hooks:** Write the native OS listeners to request raw IMU and GPS permissions, pushing the data directly into the C++ engine's shared memory buffers.
- **Visualization:** Establish a local WebSocket listener to ingest the final 10Hz map-matched JSON payload and smoothly animate a vehicle icon across an offline Mapbox layer.

3. API Data Contracts & Integration Schemas

AI to Fusion Contract (C++) & GNSS to Fusion Contract (C++)

```
struct AIPrediction {
    uint64_t timestamp_ms;
    float velocity_mps;
    float yaw_rate_rads;
};

struct GNSSUpdate {
    uint64_t timestamp_ms;
    double latitude;
    double longitude;
    float accuracy_m;
    bool is_valid;
};
```